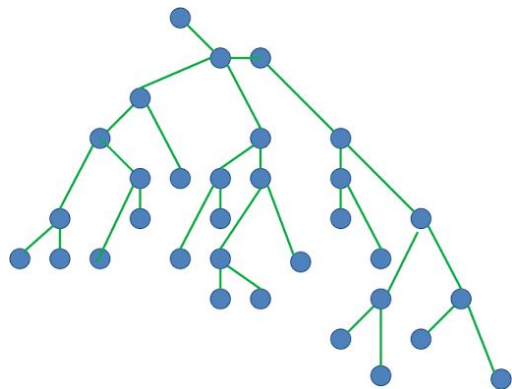




תכנות תחרותי

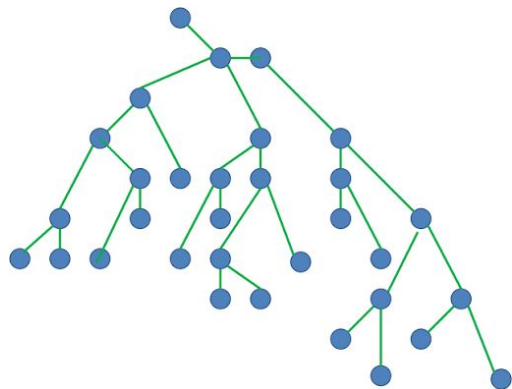
הרצאה 5



עץ - הגדרה

ישנם שני סוגי עצים; מכוונים ולא מכוונים

- עצים לא מכוונים הם פשוט גרפים קשירים חסרי מעגלים, כלומר בין כל שני צמתים בגרף קיים בדיוק מסלול פשוט אחד
- עצים מכוונים הם גרפים מכוונים בהם קיים צומת "שורש" ממנו קיים מסלול מכוון יחיד לכל שאר הצמתים בגרף



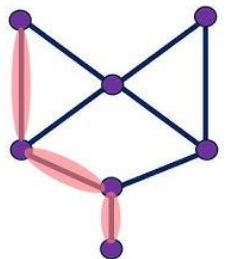
עץ - תכונות

- אם גרף G בעל n צמתים הוא עץ אז:
 - G הוא גרף קשיר
 - אין ב- G אף מעגל פשוט
 - כל קשת שנוסיף ל- G תיצור מעגל פשוט
 - אם נוריד מ- G קשת הוא יפסיק להיות קשיר
 - בין כל שני צמתים ב- G קיים מסלול פשוט יחיד
 - יש ב- G בדיוק $n-1$ קשתות

Diameter of a Graph

Diameter of a graph:

The maximum distance
between any pair of
vertices in the graph.



$\text{diameter}(G) = 3$

קוטר בעץ

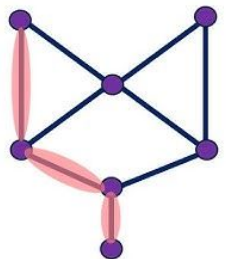
קוטר בגרף - הגדרה

- קוטר בעץ או בגרף כללי הוא אורך המסלול הארוך ביותר בגרף

Diameter of a Graph

Diameter of a graph:

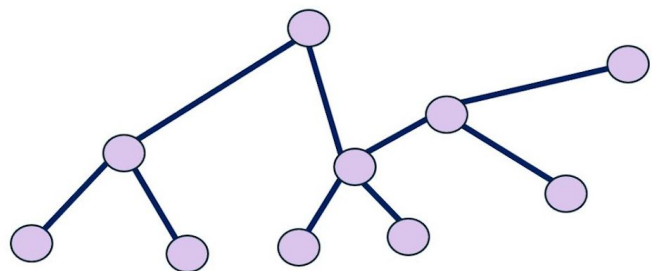
The maximum distance between any pair of vertices in the graph.



$\text{diameter}(G) = 3$

קוטר בעץ - בעיה

- בהינתן עץ, אנחנו רוצים למצוא את הקוטר שלו.
- כיצד נוכל למצוא אותו? מה הסיבוכיות?



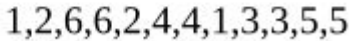
קוטר בעץ - פתרון

- הפתרון הטריטוריאלי הוא פשוט לבדוק עבור כל צומת את המרחק מכל שאר הצמתים.
- נריץ bfs מכל צומת,
- הקוטר הוא המרחק המקסימלי שנמצא.
- סיבוכיות הזמן של זה היא כמובן: $O(n^2)$
- אנחנו יכולים יותר טוב!

קוטר בעץ - פתרון

- נפתור על ידי 2 ריצות bfs בלבד!
- נבחר צומת שרירותית s , נריץ ממנה bfs ונמצא את המרחקים לכל שאר הצמתים בעץ.
- נלך לצומת הרחוק ביותר מ- s , צומת u .
- צומת u הוא בהכרח אחד הצדדים של קוטר בעץ (מסלול בעל אורך מקסימלי).
- נריץ bfs מ- u ונמצא את הצומת הכי רחוק ממנו v .
- המסלול בין u ל- v בעץ הוא קוטר.
- נשים לב שזה עובד רק כי הגרף עץ!





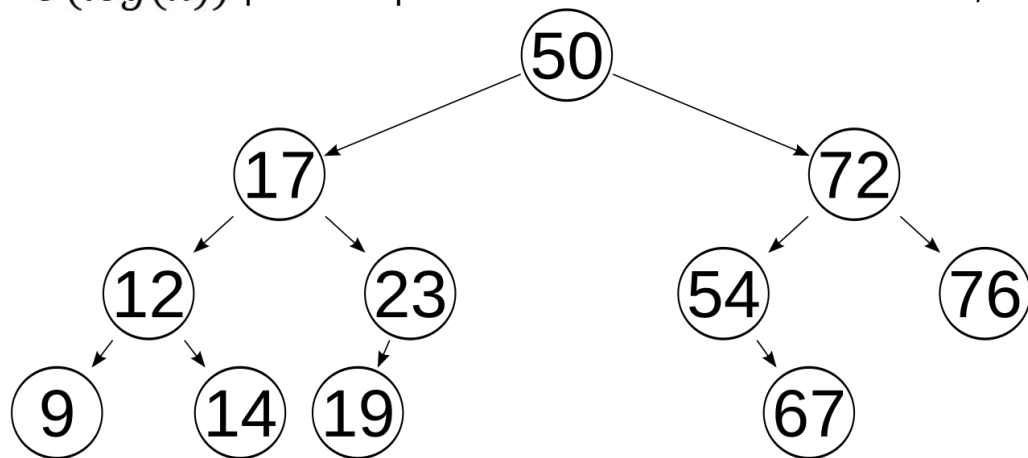
מעבר אוילר



סכום של תת עץ - בעיה

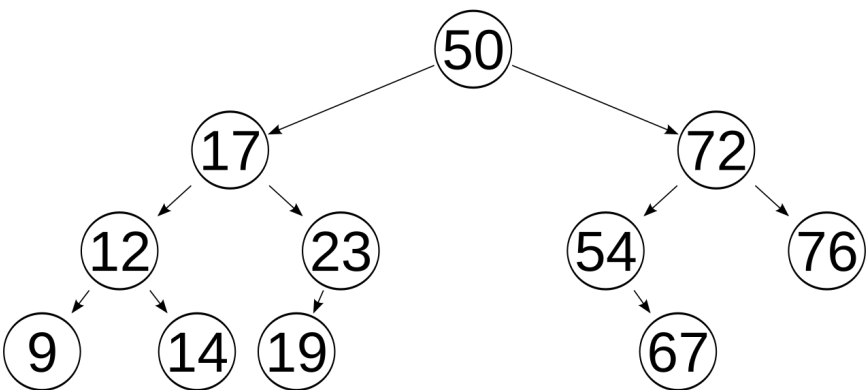
• בהינתן עץ מושרש (מכוון) בעל משקלים על הצמתים נרצה לתמוך בשאילתות הבאות:

- אתחול מבנה הנתונים. זמן $O(n)$
- בהינתן צומת u , וערך x , עדכן את ערך הצומת u ל- x . זמן $O(\log(n))$
- בהינתן צומת u , החזר את סכום ערכי הצמתים בתת העץ של u . זמן $O(\log(n))$



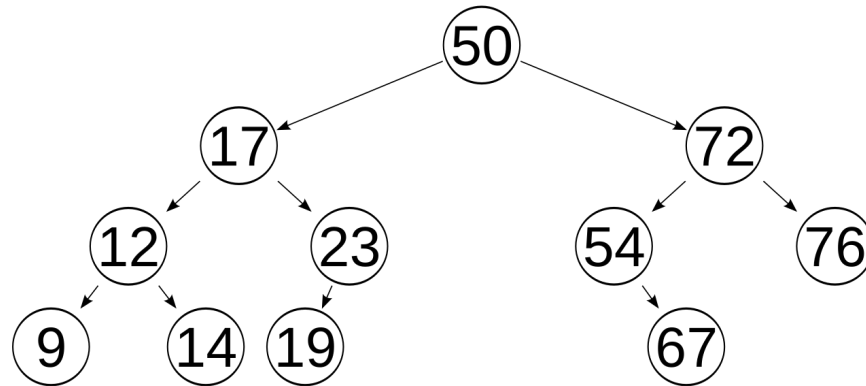
סכום של תת עץ - פתרון

- במבט ראשון הבעיה נשמעת ממש פשוטה,
- כל צומת יחזיק את סכום תת העץ של הבנים שלו, בנייה כמובן בזמן לינארי ע"י DFS
- אפשר אפילו לשמור מערך של ערכי הצמתים ולהחזיר סכום תת עץ של צומת u בזמן $O(1)$



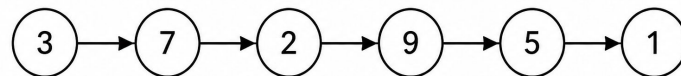
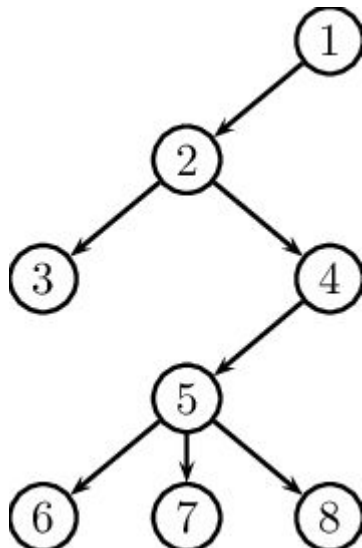
סכום של תת עץ - פתרון

- אז למה אמרנו לכם זמן $O(\log(n))$?
- עצים לא תמיד נראים ככה



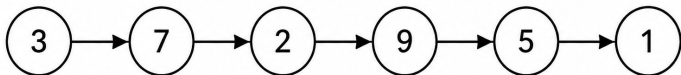
סכום של תת עץ - פתרון

- אז למה אמרנו לכם זמן $O(\log(n))$?
- לפעמים הם נראים ככה



סכום של תת עץ - פתרון

- אז למה אמרנו לכם זמן $O(\log(n))$?
- איך נעדכן ערך של צומת u ?
- נצטרך לעדכן את ערכי כל הצמתים במסלול מהשורש ל- u .
- סיבוכיות זמן $O(n)$, לא טוב מספיק אנחנו רצינו $O(\log(n))$

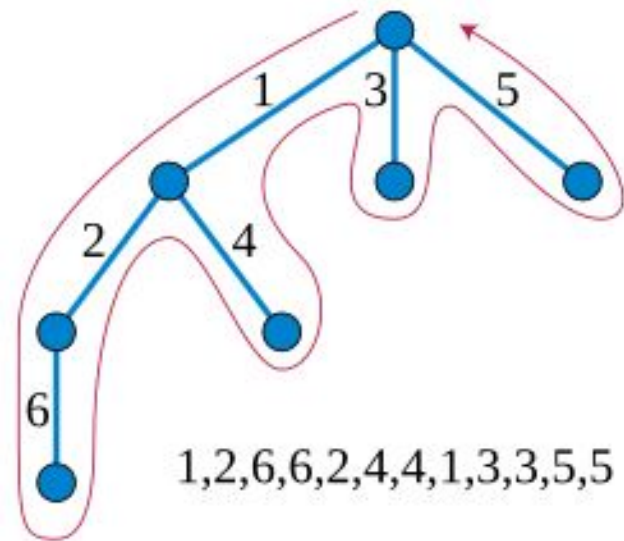


סכום של תת עץ - פתרון

- בעצם נשים לב כי שאילתה של סכום תת עץ היא שאילתה של סכום כלשהו של איברים בסיבוכיות $O(\log(n))$.
- ראינו כבר משהו דומה, segment tree!
- אבל עצי סגמנטים עובדים על מערכים לא על עצים, איך אפשר להפוך את העץ למערך?

מעבר אויילר

- מעבר אויילר הוא מעבר DFS על הגרף שבו כל צומת מקבל זמן כניסה וזמן יציאה.
- זמן כניסה ניתן לצומת ברגע הראשון שבו אנחנו רואים את הצומת, זמן יציאה ניתן לצומת אחרי שסיימנו לבקר בכלל תת העץ שלה.



מעבר אויילר - מימוש

```
void Euler_Tour(vvi &graph, int u, vector<pii> &times, int &time) {  
    times[u].first = time++; //Set entrance time for u and increment time  
  
    for (auto v : graph[u]) { //Recursively run the DFS on the subtree of u  
        Euler_Tour(&graph, v, &times, &time);  
    }  
  
    times[u].second = time++; //Set exit time and increment time  
}
```

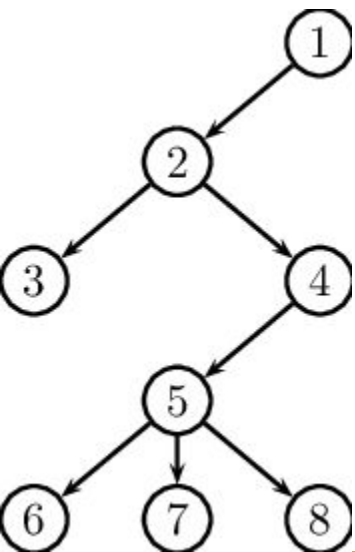
מעבר אויילר

- כל צומת u קיבל שני ערכים, $1 \leq u_1 < u_2 \leq 2n$
- בנוסף נשים לב שלכל צומת v בתת העץ של u מתקיים $u_1 < v_1 < v_2 < u_2$
- לכן כל הצמתים בתת העץ של u הם כל הצמתים שהערכים שלהם בטווח $[u_1, u_2]$

סכום של תת עץ - בעיה

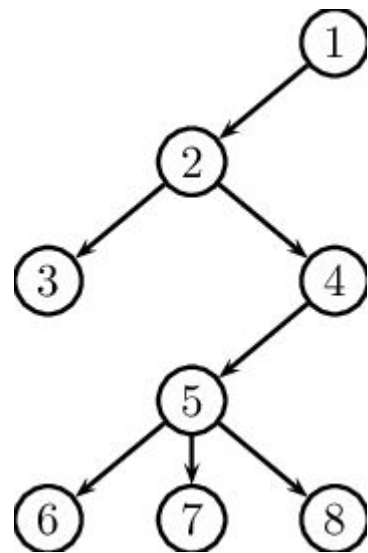
• בהינתן עץ מושרש (מכוון) בעל משקלים על הצמתים נרצה לתמוך בשאילתות הבאות:

- אתחול מבנה הנתונים. זמן $O(n)$
- בהינתן צומת u , וערך x , עדכן את ערך הצומת u ל- x . זמן $O(\log(n))$
- בהינתן צומת u , החזר את סכום ערכי הצמתים בתת העץ של u . זמן $O(\log(n))$



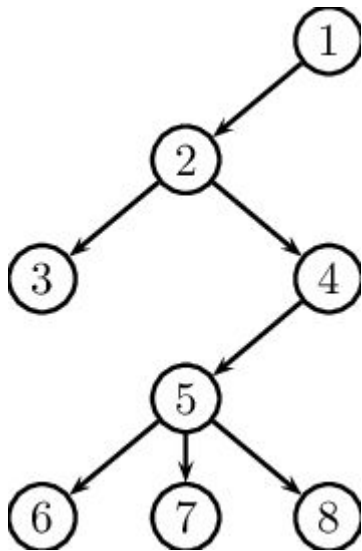
סכום של תת עץ - פתרון

- אולי עכשיו יש לכם פתרון?



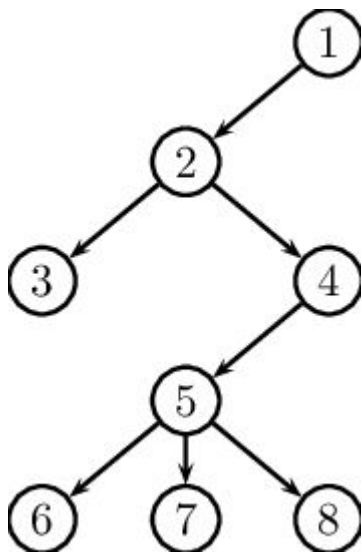
סכום של תת עץ - פתרון

- נבנה מערך A בגודל $2n$
- עבור צומת u בעלת זמני הכניסה והיציאה u_1, u_2 נגדיר $A[u_1] = value(u)$, $A[u_2] = value(u)$
- סכום איברי המערך בתחום $[u_1, u_2]$ שווה פעמיים סכום תת העץ של u !

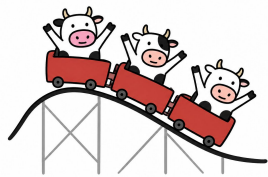


סכום של תת עץ - פתרון

- נבנה מערך A בגודל $2n$
- עבור צומת u בעלת זמני הכניסה והיציאה u_1, u_2 נגדיר $A[u_1] = value(u)$, $A[u_2] = value(u)$
- סכום איברי המערך בתחום $[u_1, u_2]$ שווה פעמיים סכום תת העץ של u !



- כל מה שנשאר זה לבנות עץ סגמנטים על A



ארץ הפרות

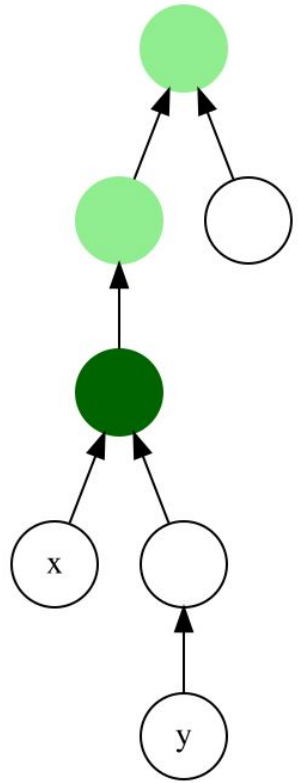
- ארץ הפרות הוא פארק שעשועים אקסטרים שנבנה במיוחד לפרות. הפרות מסתובבות בפארק ועולות על רכבות פרים שונות.
- בפארק יש N רכבות פרים ובין זוגות מסויימים של רכבות יש שבילי הליכה קצרים. בפארק סך הכל $N-1$ שבילים כך שקיים מסלול יחיד בדיוק בין כל זוג רכבות פרים.
- לכל רכבת i יש ערך הנאה $a[i]$ שיכול להשתנות במהלך היום.
- סך ההנאה של פרה שהלכה במסול מרכבת i לרכבת j הוא xor ערכי הרכבות על המסלול.
- עזרו לפרות למצוא את ערכי ההנאה של המסלולים המתוכננים שלהם, כלומר בהינתן שני צמתים u, v מצאו את ערך ההנאה בסלול $u \rightarrow v$

ארץ הפרות - פתרון

- נעבור עם Euler Tour על הגרף
- נבנה מערך A בגודל $2n$
- עבור צומת u בעלת זמני הכניסה והיציאה u_1, u_2 נגדיר $A[u_1] = value(u)$, $A[u_2] = value(u)$
- עבור צומת u , ה- xor של רישא במערך A עד המיקום u_1 או u_2 שווה לערך ה- xor של המסלול מהשורש אל u .
- נוכל לתחזק עץ סגמנטים על המערך.

roller-cow-ster





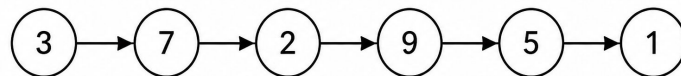
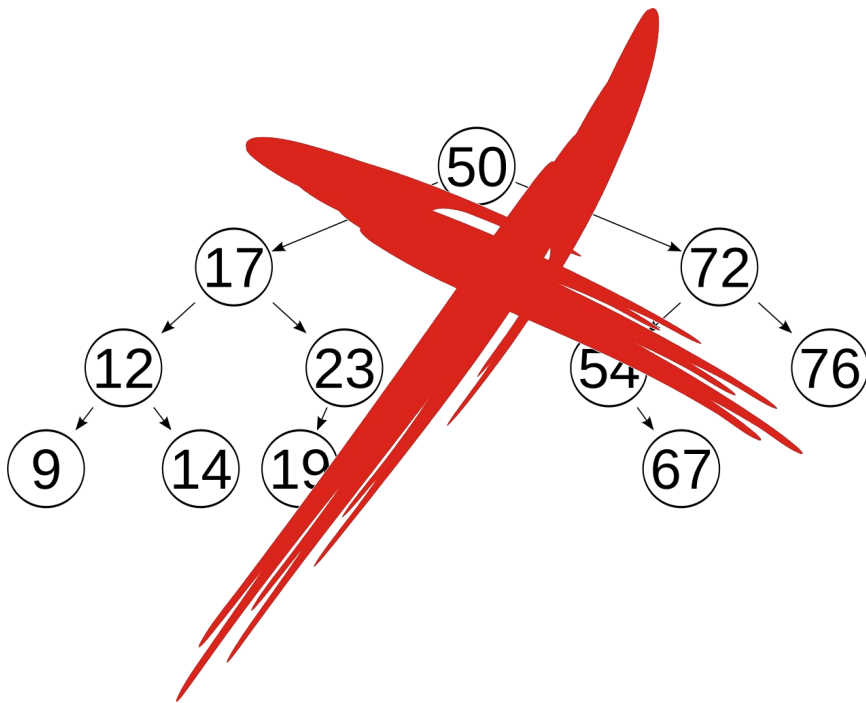
LCA - binary lifting

אב קדמון משותף - בעיה

- בהינתן עץ G מושרש בצומת s נרצה לענות על שאלות:
- בהינתן צמתים u, v החזר את האב הקדמון המשותף הנמוך ביותר של u, v
- אב קדמון משותף של שני צמתים u, v הוא צומת w כך שהמסלול בין s ל- u עובר ב- w וגם המסלול בין s ל- v עובר ב- w .
- אב קדמון משותף נמוך ביותר הוא האב הקדמון המשותף של u, v בעל העומק המקסימלי בעץ, כלומר המרחק המקסימלי מ- s .

אב קדמון משותף - פתרון

- שוב פעם אנחנו חייבים לזכור כי העצים שלנו לא חייבים להיות מאוזנים



בעיית מבנה נתונים

- בהינתן עץ מושרש נרצה לתחזק את מבנה הנתונים הבא:

- בנייה בזמן $O(n \log(n))$

- בהינתן צומת u וערך k , החזר את האב הקדמון ה- k של u . זמן $O(\log(n))$

Binary lifting

- הפתרון לבעיה הוא שכל צומת יזכור את האבות הקדמונים שלו.
- אבל לכל צומת יכולים להיות h אבות קדמונים, איזה אבות נזכור ואיך?

Binary lifting

- כל צומת יזכור את האבות הקדמונים מחזקות של 2 שלו
- כלומר האבות הקדמונים ה-1,2,4,8,...

בנייה - Binary lifting

- נוכל לבנות את לכל צומת את האבות הקדמונים שלו בזמן כולל של $O(n \log(n))$
- נתחיל מלמצוא לכל צומת את האבא הראשון שלו.
- אחר כך בכל צעד i נמצא את האבא ה- 2^i של צומת ע"י לגשת לאבא ה- 2^{i-1} שלו שמצאנו בסיבוב הקודם ואז ללכת לאבא ה- 2^{i-1} שלו.

```
void binary_lifting(){
    for(int i = 1; i < logn; i++){
        for(int j = 1; j < maxn; j++){
            if(parents[j][i-1] != -2)
                parents[j][i] = parents[parents[j][i-1]][i-1];
        }
    }
}
```

בעיית מבנה נתונים - פתרון

- כדי למצוא את האב הקדמון ה- k של צומת u נפרק את k לפירוק הבינארי שלו וכך נוכל תוך $O(\log(n))$ קפיצות לאבות קדמונים להגיע לאב קדמון ה- k של u .

אב קדמון משותף - בעיה

- בהינתן עץ G מושרש בצומת s נרצה לענות על שאלות:
- בהינתן צמתים u, v החזר את האב הקדמון המשותף הנמוך ביותר של u, v

□ אב קדמון משותף של שני צמתים u, v הוא צומת w כך שהמסלול בין s ל- u עובר ב- w וגם המסלול בין s ל- v עובר ב- w .

□ אב קדמון משותף נמוך ביותר הוא האב הקדמון המשותף של u, v בעל העומק המקסימלי בעץ, כלומר המרחק המקסימלי מ- s .

אב קדמון משותף - פתרון

- נבנה binary lifting על העץ
- בנוסף נעבור על כל העץ ונשמור לכל צומת את העומק שלו
- בהינתן שני צמתים u, v , נניח בה"כ כי העומק של u גדול מהעומק של v
- נקפוץ לאב הקדמון של u שנמצא בעומק של v
- עכשיו שני הצמתים שלנו באותו עומק, כל צומת עולה כל פעם לאב הקדמון הגבוה ביותר שלו (מבין אלו ששמורים לו ב-binary lifting) כך שהם לא אותו צומת עד שאין יותר כאלה
- כלומר האב הקדמון הראשון של שני הצמתים הוא אותו אב קדמון ולכן הוא גם ה-LCA